

Syntax Analysis

Parser

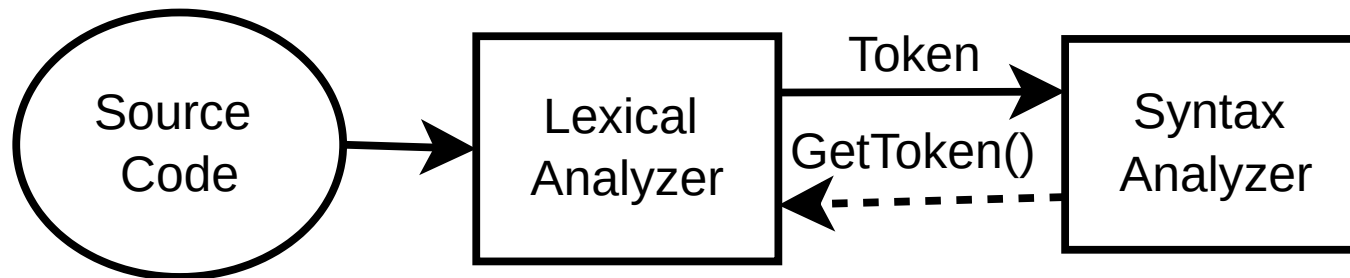
Ahmad Yoosofan

Compiler course

University of Kashan

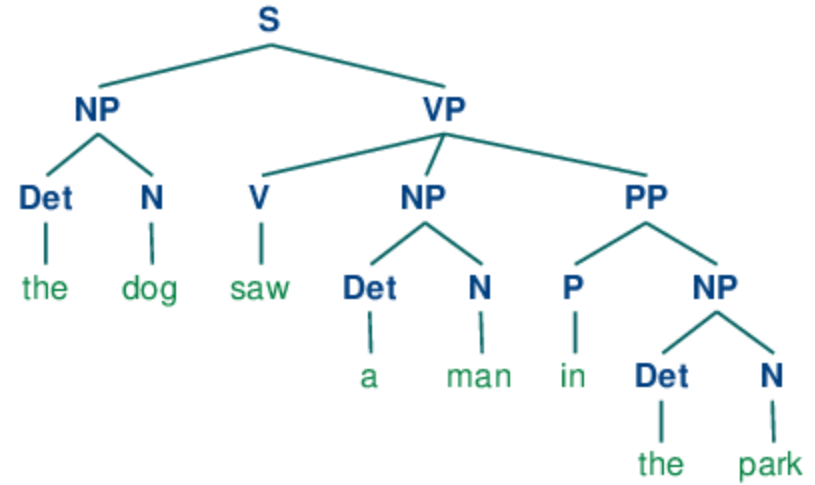
Checking Syntax

1. I am student
2. ☒ I are student
3. ☒ You am student
4. ☒ They am student
5. $342 + 261 * 5$
6. $45 * (23 + 332)$
7. ☒ $23 * + 54$
8. ☒ $2323 +$
9. ☒ $* 232$
10. ☒ $2323 + 23 * (23 (23))$
11. ☒ $234 * (45 * (23+3))$
12. We need more theory to simplify the task



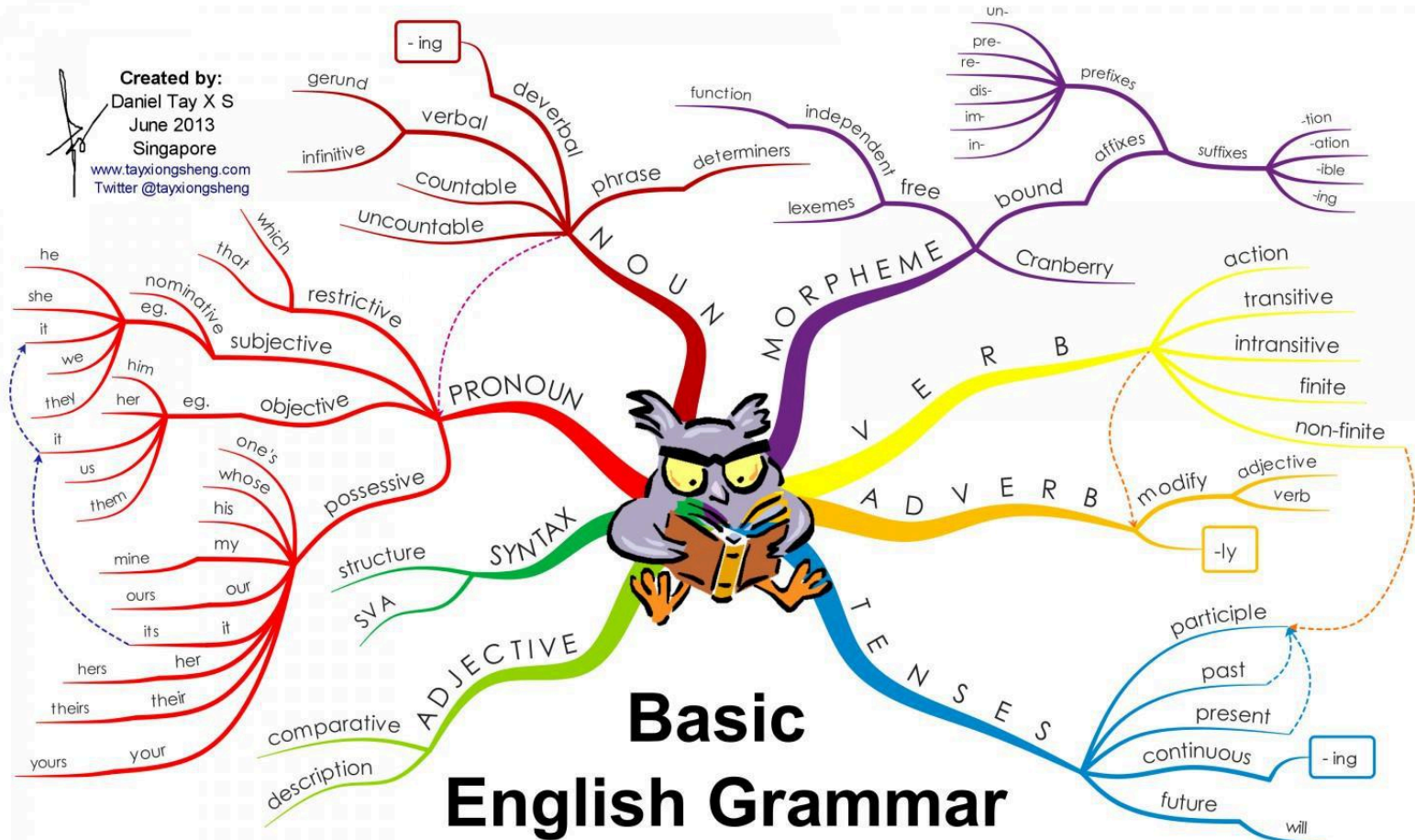
Grammar

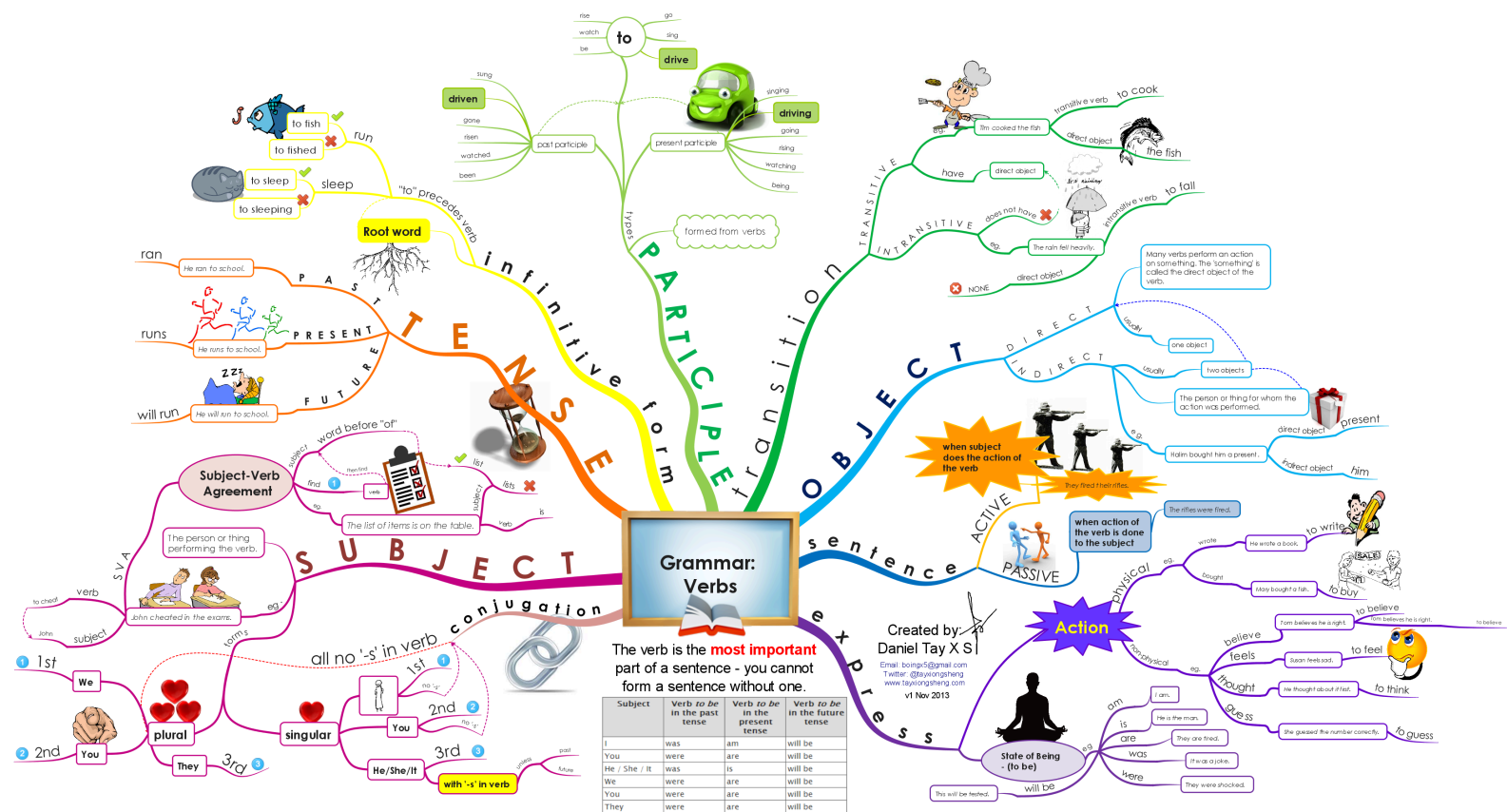
- Regular Expression Weakness
 - Nested comments `/* /* */ */`
 - Parenthesis and Braces `() {} []`
 - State machine needs memory
- context-free grammar (CFG)
- BNF (Backus-Naur Form) notation
- checks a given source program
- creates the parse tree
- Otherwise gives the error messages
- $\rightarrow, \lambda, |$
- $S \rightarrow aA$
- $S \rightarrow b$
- You can use
 - $\rightarrow$ instead of $\rightarrow$



<https://www.nltk.org/book/ch08.html>

Created by:
Daniel Tay X S
June 2013
Singapore
www.tayxiangsheng.com
Twitter @tayxiangsheng





Simple Grammars

- I

1. $S \rightarrow aSb$
2. $S \rightarrow b$
3. $abb, aabbb, aaabbbb,$

- II

- $S \rightarrow (S)$
- $S \rightarrow a$
- $a, (a), ((a))$

- III

- $S \rightarrow [A]$
- $A \rightarrow aS$
- $A \rightarrow \lambda$
- $[], [a[]], [a[a[]]]$

1. Writing grammar is harder than finding sentences of a grammar

- 87
- $78968 + (3334+43)$
- $78968 * (3334+43)$
- $78968 * (334+43)$
- $78968 * ((334+43) * 4)$
- $78968 * ((334+43)* 4)$
- $(23 + 45) + 45$
- $(23 + 45) + (45 * 443)$
- $2 * ((23 + 45) + (45 * 443))$

Grammar of simpler calculator

1. It has just +, *, (,)
2. There is no priority
3. No unary operator (+)
4. Use paranthesis for more that one operator
 1. (2+3)+4
 2. 2+(4+3)
 3. (2+(4+3))+5
 4. (2*(4*3))*5
5. Terminals
6. INT (for simplicity we use token.type: i)
 1. (i+i)+i
 2. i+(i+i)
 3. (i+(i+i))+i
 4. (i*(i*i))*i
7. {'+', '*'}
8. {'(', ')'}
9. No need for naming operators, just using the symbol
10. Use recursion

1. Start Symbol (S)
2. $S \rightarrow A + A$
3. $S \rightarrow A * A$
4. $S \rightarrow A$
5. $A \rightarrow (S)$
6. $A \rightarrow i$
7. Derivation Tree

Derivation Tree(I)

1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow i$

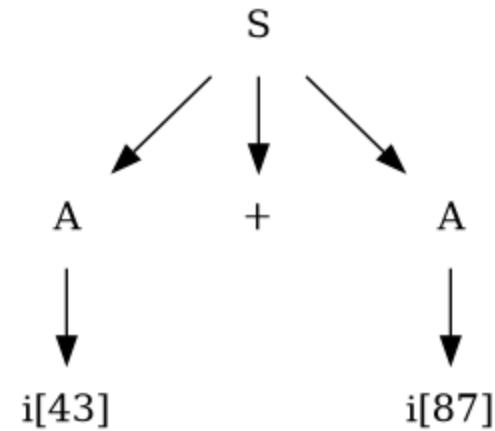
- $43 + 87$

Left Most Derivation

1. $S \Rightarrow A + A \Rightarrow$
2. $i[43] + A \Rightarrow$
3. $i[43] + i[87]$

Right Most Derivation

1. $S \Rightarrow A + A \Rightarrow$
2. $A + i[87] \Rightarrow$
3. $i[43] + i[87]$



Derivation Tree(II)

1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow i$

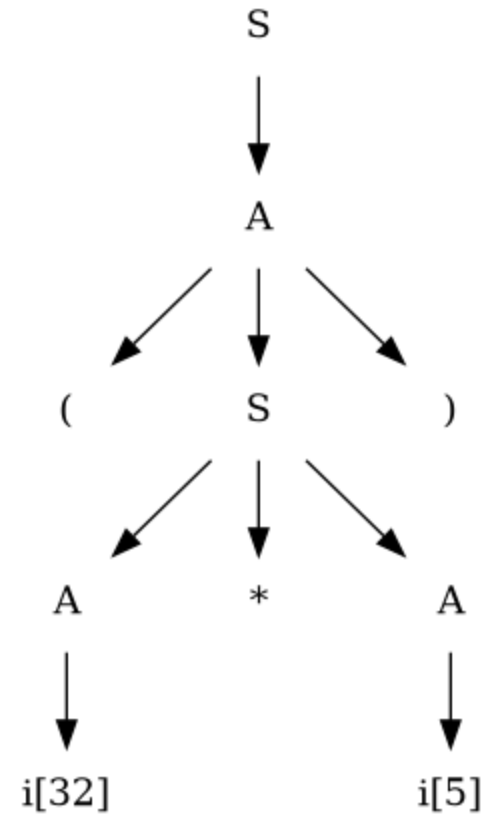
- $(32 * 5)$

Left Most Derivation

1. $S \Rightarrow A \Rightarrow$
2. $(S) \Rightarrow$
3. $(A * A) \Rightarrow$
4. $(i[32] * A) \Rightarrow$
5. $(i[32] * i[5])$

Right Most Derivation

1. $S \Rightarrow A \Rightarrow$
2. $(S) \Rightarrow$
3. $(A * A) \Rightarrow$
4. $(A * i[5]) \Rightarrow$
5. $(i[32] * i[5])$



Derivation Tree(III)

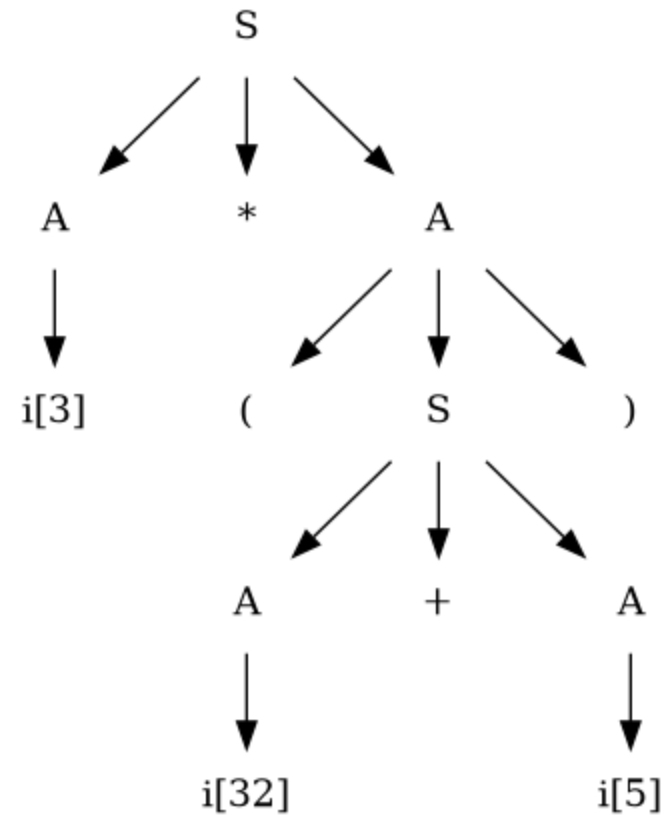
- $S \rightarrow A + A \mid A * A \mid A$
- $A \rightarrow (S) \mid i$
- $3 * (32 + 5)$

Left Most Derivation

1. $S \Rightarrow A * A \Rightarrow$
2. $i[3] * A \Rightarrow$
3. $i[3] * (S) \Rightarrow$
4. $i[3] * (A + A) \Rightarrow$
5. $i[3] * (i[32] + A) \Rightarrow$
6. $i[3] * (i[32] + i[5])$

Right Most Derivation

1. $S \Rightarrow A * A \Rightarrow$
2. $A * (S) \Rightarrow$
3. $A * (A + A) \Rightarrow$
4. $A * (A + i[5]) \Rightarrow$
5. $A * (i[32] + i[5]) \Rightarrow$
6. $i[3] * (i[32] + i[5])$



More Languages

1. Adding Priority
2. Adding Minus and Divide
3. Adding Variables (Assignment)
4. A simple Programming Language (if and while)

Next Topic: Recursive Descendant Parser

End

